KijaniKiosk Capstone - Reflection

Track A: Infrastructure-First | Giovanni Bwayo | August 14, 2026

What did you get wrong?

I planned to trigger the serverless chain's kk-processor to kk-notifier handoff through MinIO's native S3 event-notification system, on the assumption it would work as a drop-in local equivalent to real AWS S3-to-Lambda triggers. It didn't. MinIO's notify_webhook target accepted **enable=on** through mc admin config set, and the command reported success every time, but the field never actually persisted on the named target. Three separate attempts, two full container restarts, same result. I kept assuming MinIO's config-set semantics worked like Terraform's, where a reported success means the state genuinely changed. That assumption cost me close to forty-five minutes before I recognized this as a bug in the tool, not a mistake in my commands. Looking back, I would check MinIO's GitHub issues for this exact symptom after the second failed attempt, not the fourth. The fix I landed on instead, direct HTTP chaining between functions rather than event-driven triggering, turned out better than my original plan anyway. It needs no MinIO admin configuration at all, and it is the pattern this course's own materials point to as the right answer for local-first serverless development.

What is the most important thing you learned?

Git hygiene has to be built into the working rhythm from session one. It cannot be added back in later. This capstone was built in a single sitting under real time pressure, so every commit is dated the same day. The commit messages are genuinely good on their own terms: conventional format, specific rationale, honest documentation of every deviation. But the rubric's git hygiene dimension wants commits spread across at least three calendar days as evidence of iterative development, and no amount of message quality fixes that after the fact. I understood this requirement in the abstract since Week 5, when the course first introduced conventional commits and branch discipline. What I had not internalized is that the day-spread part is a structural constraint, not a stylistic one. It can only be satisfied by the calendar itself. There is no commit message clever enough to stand in for time. What actually changed in how I think about this: professional git history is not just documentation of what happened, it is evidence of a working process, and you cannot manufacture evidence of a process after the process is already over.

What would a second pass look like?

Three specific changes, not general improvements. First, a dead-letter queue or equivalent message broker between kk-receipts, kk-processor, and kk-notifier. The direct HTTP chaining that replaced the MinIO webhook approach has no retry guarantee if a downstream function is unavailable at call time, and right now that gap only exists as a line on the Production Gaps slide, not as working code. Second, I would replace the hand-provisioned Jenkins kubeconfig, manually flattened and copied into the container, working but undocumented as infrastructure-as-code, with a Kubernetes ServiceAccount token generated and injected through Terraform or Ansible. That closes the least-privilege gap the AI governance log's second entry flagged directly. Third, I would move Terraform state to the MinIO S3-compatible backend already used elsewhere in this course, instead of local state. The self-review process caught a real collision when a second clone applied against the same live cluster with its own empty state file. A shared backend fixes that failure mode specifically, not just in theory.